

BÁO CÁO PHẢN BIỆN & HOÀN THIỆN KỊCH BẢN KIỂM THỬ (HUMAN REVIEW & FIX REPORT)

Mã bài tập: HW05 – Performance Testing (Exercise ID: HW05-AI)

Nhiệm vụ: Task 1 – Human Review & Fix (Mục 6 đề bài)

Sinh viên thực hiện: BaoBeiii – MSSV: 23127327

Hệ thống kiểm thử (SUT): EShop System (eshop-sut)

Công cụ AI được kiểm toán: Google Antigravity / Gemini 3.7 Flash (High)

Ngày thực hiện: 2026-09-02

1. Tóm Tắt Đánh Giá Tổng Quan (Executive Summary)

Trong Giai đoạn 1, trợ lý AI đã hỗ trợ sinh ra khung kịch bản kiểm thử ban đầu cho cả 3 kịch bản chính (**Load, Stress, Spike**) và bài kiểm tra độ bền (**Endurance**). Dù kịch bản của AI hoàn thành được cấu trúc luồng tuần tự cơ bản, quá trình **Human Review (Đánh giá của con người)** thông qua việc đối chiếu trực tiếp với mã nguồn thực tế (eshop-sut/backend/server.js) và đặc tả hệ thống đã phát hiện **6 nhóm sai sót và thiếu hụt cốt lõi** mang tính hệ thống của AI.

Tài liệu này ghi lại chi tiết các lỗi AI đã mắc phải, phân tích nguyên nhân gốc rễ (Root Cause Analysis) dưới 3 chiều kích theo đúng yêu cầu đề bài: **Chất lượng câu lệnh (Prompt Quality)**, **Hạn chế của mô hình (Model Limitations)**, và **Đặc thù của Endpoint/Hệ thống (Endpoint Characteristics)**, đồng thời cung cấp mã nguồn đã được sửa đổi và hoàn thiện.

2. Bảng Tổng Hợp So Sánh Trước & Sau Human Review

Hạng mục Đánh giá	Bản Sinh Ban Đầu của AI (Initial AI Output)	Bản Hoàn Thiện Sau Human Review (Fixed by Human)	Phân Loại Nguyên Nhân
1. Định tuyến Endpoint	Đoán mò theo quy ước REST chung: <code>/api/orders</code> , <code>/api/coupons/apply</code> , <code>/api/orders</code> .	Sửa chính xác 100% theo <code>server.js</code> : <code>/api/checkout</code> , <code>/api/apply-coupon</code> , <code>/api/orders/my-orders</code> .	Model Limitations (Pattern Bias)
2. Thời gian dừng (Think-time)	Gán cứng cố định: <code>sleep(1)</code> hoặc <code>sleep(0.5)</code> . Gây hiện tượng dồn tải đồng bộ phi thực tế.	Phân phối ngẫu nhiên mô phỏng người thật: <code>sleep(Math.random() * 2 + 1)</code> ($1.0\text{s} - 3.0\text{s}$).	Prompt Quality & Modeling
3. Độ sâu của Assertion	Nông (Shallow): Chỉ kiểm tra <code>r.status === 200</code> . Bỏ sót hoàn toàn lỗi dữ liệu bên trong payload.	Sâu (Deep): Kiểm tra cấu trúc JSON, bắt lỗi giảm giá âm (BUG-01) và bắt sai kiểu dữ liệu (BUG-03).	Model Limitations
4. Xử lý Lockout (FR-02)	Giả định đăng nhập luôn thành công; không bắt mã HTTP 403 khi bị khóa tài khoản.	Thiết lập tài khoản riêng biệt; kiểm tra bắt HTTP 403 khi chạy Stress Test để đo điểm gãy bảo mật.	Endpoint Characteristics
5. Nhận thức Nghẽn SQLite	Coi backend là Stateless vô hạn; không bắt lỗi nghẽn ghi đồng thời <code>SQLITE_BUSY</code> (HTTP 500).	Bổ sung check bắt lỗi 500 SQLite lock tại bước Checkout dưới tải cực lớn (Stress 200 VUs).	Endpoint Characteristics
6. Trình xuất Báo cáo	Không có hàm <code>handleSummary</code> ; chỉ in console log mặc định của k6.	Tích hợp thư viện <code>k6-reporter</code> xuất file HTML Dashboard tương tác trực quan tại <code>results/</code> .	Engineering Best Practice

3. Phân Tích Chi Tiết 6 Lỗi Cốt Lỗi của AI & Nguyên Nhân Gốc Rễ

❌ Lỗi 1: Ảo giác Đường dẫn API (API Endpoint Hallucination / Convention Bias)

- Hiện tượng: AI tự động giả định các endpoint theo tiêu chuẩn RESTful phổ biến trên mạng:
 - Giả định gửi đơn hàng tại: `POST /api/orders`
 - Giả định kiểm tra mã giảm giá tại: `POST /api/coupons/apply`
 - Giả định xem lịch sử đơn hàng tại: `GET /api/orders`
- Thực tế trong mã nguồn SUT (`backend/server.js`):
 - Dòng 297: `app.post("/api/checkout", ...)`
 - Dòng 363: `app.post("/api/apply-coupon", ...)`
 - Dòng 311: `app.get("/api/orders/my-orders", ...)`

- **Nguyên nhân gốc rễ (Root Causes):**

- i. *Model Limitations*: Các mô hình LLM được huấn luyện trên hàng triệu dự án mã nguồn mở và có xu hướng "khớp mẫu" (pattern matching) theo quy ước RESTful phổ thông (`/resources/action` hoặc `/resources`), dẫn đến hiện tượng suy diễn cảm tính khi chưa đọc file router cụ thể.
- ii. *Prompt Quality*: Câu lệnh ban đầu mô tả luồng nghiệp vụ ("Checkout", "Coupon", "Orders") nhưng không đính kèm toàn bộ file `server.js` vào ngữ cảnh context.

❌ Lỗi 2: Thời Gian Dừng Cứng Nhắc & Thiếu Tính Ngẫu Nhiên (Flat Think-Time)

- **Hiện tượng**: AI sử dụng lệnh `sleep(1);` hoặc `sleep(0.5);` cố định giữa mọi bước trong hàm `default function()`.
- **Hậu quả Kỹ thuật**: Trong kiểm thử hiệu năng, việc dùng think-time phẳng (flat) khiến hàng chục Virtual Users (VUs) được kích hoạt cùng lúc sẽ thực hiện các request hoàn toàn đồng nhịp (lock-step synchronization), tạo ra các sóng xung kích nhân tạo (micro-spikes) không phản ánh đúng hành vi người dùng thực tế.
- **Khắc phục sau Human Review**: Áp dụng phân phối thời gian dừng ngẫu nhiên:

```
// Người dùng dừng đọc màn hình từ 1 đến 3 giây ngẫu nhiên
sleep(Math.random() * 2 + 1);
```

- **Nguyên nhân gốc rễ: Prompt Quality & Model Limitation**: AI có xu hướng tối thiểu hóa mã mẫu để ngắn gọn, thường mặc định `sleep(1)` như một placeholder mà bỏ qua nguyên lý thống kê của kỹ thuật kiểm thử tải.

❌ Lỗi 3: Kiểm Tra Nông (Weak / Shallow Assertions)

- **Hiện tượng**: AI chỉ kiểm tra điều kiện bề mặt: `check(res, { 'status is 200': (r) => r.status === 200 })`.
- **Hậu quả Kỹ thuật**: Trong SUT EShop, có rất nhiều lỗi logic nghiêm trọng trả về mã HTTP 200 nhưng nội dung bên trong hoàn toàn sai lệch:
 - i. *BUG-01*: API `/api/apply-coupon` tính sai công thức giảm giá phần trăm `Math.floor(total_amount * (1 - coupon.discount_value))`, khiến giảm 10% biến thành tăng gấp 10 lần giá tiền, nhưng vẫn trả về HTTP 200!
 - ii. *BUG-03*: API `/api/products/:id` trả về `price` dạng String `"350000"` đối với ID chẵn, vẫn trả về HTTP 200! Nếu chỉ check HTTP 200 như AI, bộ test sẽ báo xanh (Pass 100%) và che giấu hoàn toàn các lỗi nghiêm trọng này.

- **Khắc phục sau Human Review:** Thêm các Deep Assertions:

```
check(couponRes, {
  'Coupon discount valid (Detect BUG #1)': (r) => {
    if (r.status === 200 && r.json('success')) {
      return r.json('final_amount') < totalAmount && r.json('discount_amount') > 0;
    }
    return true;
  }
});

check(prodDetailRes, {
  'Detect BUG #3 (Price must be number)': (r) => typeof r.json('price') === 'number';
});
```

❌ Lỗi 4: Bỏ Sót Cơ Chế Khóa Tài Khoản (Missing Account-Lockout Handling FR-02)

- **Hiện tượng:** AI cho tất cả các VU dùng chung tài khoản hoặc lấy modulo đơn giản `users[__VU % users.length]`. Khi chạy Stress Test có chứa tài khoản thử nghiệm lỗi, cơ chế FR-02 trong `server.js` kích hoạt: tài khoản bị khóa trong 3 phút và trả về mã `403 Forbidden`. AI không có cơ chế bắt mã 403, khiến toàn bộ các request tiếp theo của VU đó (Browse, Checkout) bị lỗi dây chuyền do thiếu Token.
- **Nguyên nhân gốc rễ: Endpoint Characteristics:** Cơ chế khóa tài khoản dựa trên trạng thái nội tại (Stateful Database Column: `login_attempts`, `locked_until`) là một đặc thù bảo mật ít gặp trong các API REST mẫu không trạng thái mà AI hay xử lý.

❌ Lỗi 5: Thiếu Nhận Thức về Nghẽn Khóa CSDL SQLite (SQLite Concurrency Blindness)

- **Hiện tượng:** AI thiết lập ngưỡng Stress Test lên tới 200 VUs với yêu cầu `http_req_failed: ['rate<0.05']` (lỗi dưới 5%).
- **Thực tế:** SQLite sử dụng cơ chế khóa mức tập tin (File-level locking). Khi hàng chục kết nối đồng thời cùng ghi vào bảng `orders` và `coupon_usage` tại bước Checkout, SQLite sẽ văng lỗi `SQLITE_BUSY: database is locked` và Express.js trả về HTTP 500. Việc AI kỳ vọng tỷ lệ lỗi $< 5\%$ ở mức 200 VUs là hoàn toàn bất khả thi đối với kiến trúc SQLite mặc định.
- **Khắc phục sau Human Review:** Điều chỉnh ngưỡng chấp nhận lỗi trong Stress Test lên `rate < 0.25` và cài đặt check bắt lỗi `Detect SQLite DB lock / 500 error under high`

❌ Lỗi 6: Thiếu Tích Hợp Báo Cáo HTML Trực Quan (Interactive HTML Dashboard)

- **Hiện tượng:** Kịch bản k6 của AI không có hàm xuất dữ liệu tổng kết `handleSummary(data)` . Khi chạy xong, kết quả chỉ hiển thị trên màn hình dòng lệnh (stdout) và biến mất, không tạo ra các tài liệu kiểm chứng lưu trữ độc lập theo yêu cầu nộp bài của đề bài.
- **Khắc phục sau Human Review:** Tích hợp hàm `handleSummary(data)` với thư viện `k6-reporter` để tự động tạo file `summary.html` có biểu đồ tương tác cao cấp ngay trong thư mục `results/` .

4. Minh Họa So Sánh Code Trước & Sau Sửa Đổi (Code Diff)

```
--- AI_Initial_Load_Test.js
+++ Human_Refined_Load_Test.js
@@ -17,7 +17,10 @@
-// AI: Dùng think-time cố định 1s
-sleep(1);
+// HUMAN: Phân phối thời gian dừng ngẫu nhiên người thật (1s - 3s)
+sleep(Math.random() * 2 + 1);

-// AI: Đoán sai endpoint, thiếu total_amount và user_id
-http.post(`${BASE_URL}/api/coupons/apply`, JSON.stringify({ code: "SAVE10" }));
+// HUMAN: Đúng endpoint thực tế, truyền đủ tham số và cài bẫy phát hiện BUG-01
+const couponRes = http.post(`${BASE_URL}/api/apply-coupon`, JSON.stringify({
+  code: orderInfo.coupon_code,
+  total_amount: totalAmount,
+  user_id: userId
+}));
+check(couponRes, {
+  'Detect BUG #1 (Negative discount)': (r) => r.json('final_amount') < totalAmount
+});

-// AI: Đoán sai endpoint checkout
-http.post(`${BASE_URL}/api/orders`, ...);
+// HUMAN: Đúng endpoint checkout thực tế
+http.post(`${BASE_URL}/api/checkout`, ...);
```

5. Kết Luận Bài Học Rút Ra từ Human Review (Key Takeaways)

1. **AI là công cụ tăng tốc tuyệt vời, nhưng không thể thay thế kỹ sư kiểm thử:** AI giúp dựng nhanh khung kịch bản, cú pháp k6/JMeter và cấu hình stages trong vài giây. Tuy nhiên, nếu không có sự rà soát tỉ mỉ của con người đối chiếu với mã nguồn backend, kịch bản sẽ chạy vào các endpoint không tồn tại và che giấu toàn bộ lỗi của hệ thống.
2. **Giá trị của Deep Assertions:** Kiểm thử hiệu năng không chỉ là đo tốc độ (RPS, Latency) mà còn phải đảm bảo tính toàn vẹn dữ liệu dưới áp lực tải. Việc thay thế các assertion nông (status 200) bằng assertion kiểm tra giá trị nghiệp vụ đã giúp phát hiện ra các bug nghiêm trọng nhất của ứng dụng.